RESEARCH PREPRINT • INFORMATION SECURITY & WEB TECHNOLOGIES OPEN ACCESS PREPRINT

Privacy-Preserving Client-Side Information Processing: A Browser-Based Architecture for Local Data Analysis and Secure File Handling

Islam Ayoub

Independent Researcher, Information Technology

Research Area: Information Technology, Information Security, Web Security, Privacy-Preserving Computing

ABSTRACT

The increasing use of web-based applications for handling documents, images, personal records, and other forms of sensitive information has created important challenges concerning data privacy and information security. Conventional web architectures frequently require users to upload data to remote servers before processing can take place. Although server-side processing provides substantial computational capabilities, transmitting sensitive information outside the user's device introduces additional security, privacy, compliance, and data-governance considerations.

This paper investigates a client-side processing architecture in which selected data-processing operations are performed locally inside a modern web browser. The proposed approach considers the use of JavaScript, Web Workers, WebAssembly, browser-native cryptographic interfaces, and local browser capabilities to process files without transmitting their contents to an application server. Particular attention is given to document processing, image transformation, cryptographic hashing, and other computational tasks that can be performed within the browser execution environment.

The study presents an architectural model for privacy-preserving browser applications and examines its security properties, performance considerations, memory-management challenges, and limitations. The analysis suggests that client-side processing can reduce unnecessary data transmission and can provide a useful privacy-enhancing design pattern when the application does not require centralized data storage or server-side computation. However, local processing should not be interpreted as an absolute security guarantee. Browser applications remain dependent on the integrity of delivered JavaScript code, third-party dependencies, browser security mechanisms, endpoint security, and appropriate application design.

The paper concludes by proposing a practical architecture that combines local processing with explicit data-flow controls, restrictive security policies, cryptographic verification, isolated background computation, and minimal server communication.

Keywords: Information Security, Web Security, Client-Side Computing, Data Privacy, WebAssembly, Web Workers, Web Cryptography API, Local Processing, Browser Security, Privacy-Preserving Computing.

1. Introduction

Web applications have become an important platform for performing tasks that historically required dedicated desktop software. Users can now manipulate documents, convert images, calculate cryptographic hashes, analyze datasets, and perform numerous other computational operations directly through a web browser.

Despite these developments, many web applications continue to follow a conventional client-server architecture in which a user selects a file, the browser uploads it to a remote server, the server performs the required computation, and the resulting output is subsequently returned to the browser.

This architecture is practical for many applications, particularly when computation requires centralized resources or when data must be shared among multiple users. However, it also creates a data-flow problem: information that could potentially be processed locally must first leave the user's device.

For non-sensitive data, this may have little practical significance. For confidential documents, research data, financial records, personal information, intellectual property, or other sensitive material, however, unnecessary transmission can increase the number of systems and communication channels involved in processing.

An alternative approach is client-side processing, in which the browser performs computational operations locally. Modern browsers provide increasingly capable programming environments, including Web Workers for background computation, WebAssembly for high-performance execution, and the Web Cryptography API for standardized cryptographic operations. The Web Cryptography API provides browser interfaces for operations including hashing, encryption, decryption, signing, verification, key generation, and related cryptographic functions [1].

> ***Central Research Question:*** *To what extent can modern browser technologies support secure and practical local processing of sensitive information while minimizing unnecessary transmission of user data to remote servers?*

The objective is not to claim that browser-based processing eliminates all security risks. Rather, the objective is to identify an architecture in which data transmission is minimized by design and to examine the security assumptions that such an architecture requires.

1.1 Implementation Context

The architectural concepts discussed in this study are relevant to browser-based document-processing applications, including Utilvo, a web application focused on client-side document processing and file conversion. Utilvo is referenced as an implementation context for discussing local file handling, data minimization, and browser-based computation. This reference does not constitute an independent experimental validation of the security properties discussed in this study.

Further information about the application is available at https://utilvo.com/ [9].

2. Background

2.1 Traditional Client-Server Processing

A simplified conventional workflow can be represented as:

```
[CONVENTIONAL CLIENT-SERVER DATA FLOW]
User → Browser → Remote Server → Processing → Remote Server → Browser → User
```

For example, consider a document-conversion service. The user selects a PDF document, the browser uploads the document, the server performs conversion, and the resulting file is downloaded. Although this approach is technically straightforward, the original document becomes available to the server during processing.

The security implications depend on many factors, including:

- Transport-layer security;
- Server authentication and authorization;
- Server-side access controls;
- Temporary file management;

- Logging;
- Backups;
- Third-party infrastructure;
- Administrative access;
- Retention policies; and
- Potential compromise of the processing infrastructure.

Consequently, reducing unnecessary transmission can represent an additional privacy layer.

2.2 Client-Side Processing

A client-side architecture fundamentally changes the data flow:

```
[CLIENT-SIDE PRIVACY-PRESERVING DATA FLOW]
User → Browser → Local Processing (RAM Sandbox) → User
```

The server may still deliver the web application itself, but the sensitive input can remain inside the browser's execution environment. This distinction is critical.

A system that performs processing locally is not necessarily a system that has no network communication. The browser may still communicate with the server to retrieve HTML, JavaScript, CSS, fonts, libraries, analytics resources, or application configuration.

> ***Architectural Axiom:*** *Sensitive user content should not be transmitted to remote infrastructure unless transmission is explicitly required by the application's functionality.*

This distinction avoids the misleading assumption that "client-side" automatically means "offline" or "completely disconnected."

3. Proposed Architecture

The proposed architecture consists of six conceptual layers designed to isolate sensitive data from outbound transmission channels:

Layer 1: User Interface

The user interface provides mechanisms for selecting files, configuring processing options, initiating computation, displaying progress, displaying results, and exporting processed information. The interface itself should avoid unnecessary access to the raw binary content being processed.

Layer 2: Local Data Boundary

The selected file enters the browser through local browser APIs under an explicit containment pipeline:

```
[LOCAL IN-BROWSER CONTAINMENT BOUNDARY]
User File → File / Blob API → ArrayBuffer → Local Processing Pipeline (No Server Egress)
```

The application should define an explicit boundary between input acquisition and processing.

Layer 3: Background Processing

Computationally expensive operations can be transferred to a Web Worker. Web Workers provide a mechanism for executing JavaScript in a background context and communicating with the main page through message passing [2].



This architecture separates user-interface responsibilities from computational responsibilities.

4. Web Workers and Responsiveness

Processing large binary objects directly on the main browser thread can interfere with interface responsiveness. Operations involving large documents, image transformations, compression, parsing, or cryptographic computation can consume significant CPU resources.

When such operations are executed synchronously on the main thread, the browser may be unable to process user-interface tasks at the desired frequency. Web Workers provide a mechanism for moving computational work away from the main execution context [2].

A typical implementation follows this architectural pattern:

```
// Main Execution Thread
const worker = new Worker("processor.js");

worker.postMessage({
    operation: "hash",
    buffer: fileBuffer
});

worker.onmessage = (event) => {
    console.log("Processing completed:", event.data);
};
```

The dedicated worker performs computation independently without stalling the event loop:

```
// Background Worker (processor.js)
self.onmessage = async (event) => {
    const { operation, buffer } = event.data;

    if (operation === "hash") {
        const digest = await crypto.subtle.digest(
            "SHA-256",
            buffer
        );

        self.postMessage(digest);
    }
};
```

The important architectural principle is that computation occurs without transferring the original document to an external processing server.

5. WebAssembly for Local Computation

JavaScript is highly capable, but some computational workloads benefit from WebAssembly. WebAssembly is a portable compilation target designed for efficient execution in web environments. Its specification defines a low-level execution model, while browser environments provide APIs for loading and executing WebAssembly modules [3].

This makes WebAssembly particularly relevant for applications involving:

- Document parsing and compilation;
- Lossless and lossy compression;
- Image and vector processing;
- Multimedia encoding and decoding;
- Scientific computation and numerical algorithms;
- Data transformation pipelines; and
- Specialized cryptographic or mathematical libraries.

```
[NATIVE HYBRID EXECUTION MODEL]
User File → Browser Memory → Web Worker → [ JavaScript Glue ⇔ WebAssembly Module ] → Processed Output
```

The use of WebAssembly does not itself guarantee security. A vulnerable WebAssembly module, unsafe memory handling, malicious dependency, or compromised application delivery mechanism can still create security risks. Therefore, WebAssembly should be regarded as a computational technology rather than a complete security mechanism.

6. Browser-Native Cryptography

Cryptographic operations represent another important application of client-side processing. The Web Cryptography API provides standardized browser interfaces for cryptographic operations, including digest calculations, encryption and decryption, digital signatures, verification, and key management [1].

For example, SHA-256 hashing can be performed locally with zero server reliance:

```
async function calculateSHA256(file) {
    const buffer = await file.arrayBuffer();

    const hashBuffer = await crypto.subtle.digest(
        "SHA-256",
        buffer
    );

    return Array.from(new Uint8Array(hashBuffer))
        .map(byte => byte.toString(16).padStart(2, "0"))
        .join("");
}
```

The resulting digest can be utilized for file integrity verification, duplicate detection, content identification, local validation, and comparison against known manifests.

Importantly, a hash is not equivalent to encryption. Hashing is generally a one-way transformation intended for integrity and related purposes, whereas encryption is designed to protect confidentiality through reversible cryptographic transformation when the appropriate key is available. This distinction is fundamental when designing privacy-preserving applications.

7. Local File Processing and Data Minimization

A central principle of the proposed architecture is data minimization. If a user wants to resize an image, for example, the application may not need to transmit the original image to a remote server.

```
[IN-BROWSER MEDIA TRANSFORMATION PIPELINE]
Original Image → Browser Memory → Local Canvas / WASM → Transformed Image → User Download
```

Operation Type	Potential Local In-Browser Implementation	Network Requirement
Image Resizing & Cropping	HTML5 Canvas / Browser Image APIs / OffscreenCanvas	Zero Transmission
Image Format Conversion	Canvas API / WebAssembly (libvips, mozjpeg, libwebp)	Zero Transmission
SHA-256 / SHA-512 Verification	Web Cryptography API (SubtleCrypto)	Zero Transmission
Document Parsing & Merging	JavaScript / WebAssembly (pdf-lib, pdf.js, mutool)	Zero Transmission
File Compression (ZIP/GZIP)	Compression Streams API / fflate / WebAssembly	Zero Transmission
Tabular Data Transformation	JavaScript DataFrames / Web Workers	Zero Transmission
Byte-Level File Validation	Binary Magic Number Signatures / ArrayBuffer	Zero Transmission

The objective is not to move every possible operation into the browser. Some workloads inherently require server-side infrastructure. Instead, the architectural objective is to identify operations for which remote processing is unnecessary.

8. Threat Model

A privacy-preserving architecture must be grounded in an explicit threat model that accounts for the hostile nature of untrusted network and execution environments.

8.1 Network Exposure

If sensitive files never leave the device, an attacker monitoring network traffic does not receive the original file through the application's processing channel. However, other application information may still be transmitted. Therefore, network minimization should be considered separately from complete network isolation.

8.2 Malicious or Compromised Dependencies

A significant challenge for client-side applications is that the browser executes code delivered to the user's device. Third-party JavaScript libraries therefore become part of the application's security boundary. OWASP specifically identifies risks associated with client-side applications and third-party code, including sensitive data exposure and insufficient client-side security controls [4]. Consequently, minimizing dependencies is itself a critical security consideration.

8.3 Cross-Site Scripting (XSS)

A client-side application may process sensitive data locally, but an XSS vulnerability can potentially allow malicious JavaScript to interact with application data. Therefore, local processing must be fortified with output encoding, strict input validation, Content Security Policy, dependency management, secure DOM manipulation, restrictive permissions, and appropriate browser security mechanisms. OWASP describes Content Security Policy as a defense-in-depth mechanism for limiting the execution and loading of potentially malicious scripts [5].

9. Browser Storage and Privacy

One common misconception is that storing information inside a browser automatically makes the information secure. This is not necessarily true.

Browser storage mechanisms include:

- Local Storage;
- Session Storage;
- IndexedDB;
- Cookies; and
- Origin Private File System (OPFS).

OWASP guidance explicitly warns against treating browser storage as a secure location for sensitive credentials and confidential information, particularly because client-side code can access such data and XSS vulnerabilities can significantly increase exposure [6, 7].

For applications whose purpose is temporary file transformation, a preferable strategy is to minimize persistent storage altogether:

```
[ZERO-PERSISTENCE IN-MEMORY LIFECYCLE]
File Selected → Temporary RAM Buffer → Local Computation → Export Generated → Memory Cleared (GC)
```

The application should avoid retaining the original input longer than strictly necessary.

10. Memory Management

Memory management represents one of the major technical challenges in browser-based processing. Large files can require substantial memory when represented as File, Blob, ArrayBuffer, typed arrays, decoded image buffers, or WebAssembly linear memory allocations.

Furthermore, a processing pipeline may temporarily maintain several representations of the same information:

```
[MEMORY FOOTPRINT MULTIPLICATION EXAMPLE]
10 MB Compressed File
   |
   +--> ArrayBuffer: 10 MB
   |
   +--> Typed Array: Additional View (10 MB)
   |
   +--> Decoded Image (RGBA Uncompressed): Up to 80-120 MB
   |
   +--> Processed Output Buffer: 10 MB
```

Therefore, efficient memory management requires:

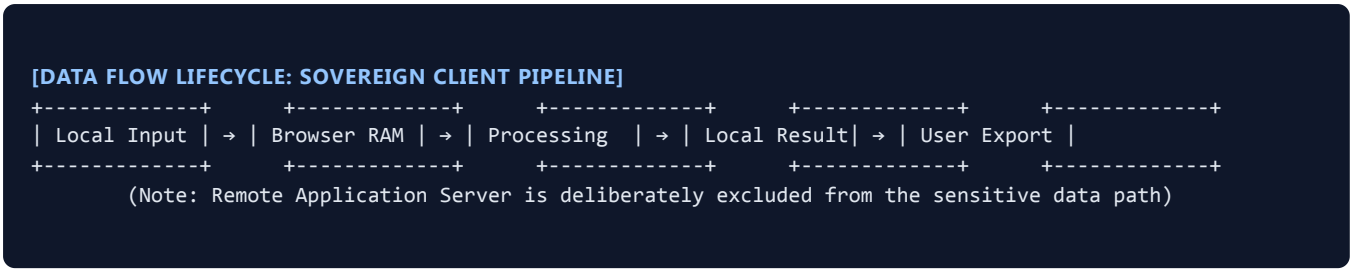
- Avoiding unnecessary array and buffer duplications;
- Processing large files in manageable chunks or stream units where possible;
- Explicitly releasing object references (setting to `null`) after execution;
- Avoiding persistent in-memory caching across distinct jobs;
- Limiting concurrent background worker allocations; and
- Actively profiling heap growth and garbage collection pauses during development.

Garbage collection is managed by the JavaScript runtime engine and must not be treated as a deterministic manual memory-deallocation mechanism.

11. Data Lifecycle Model

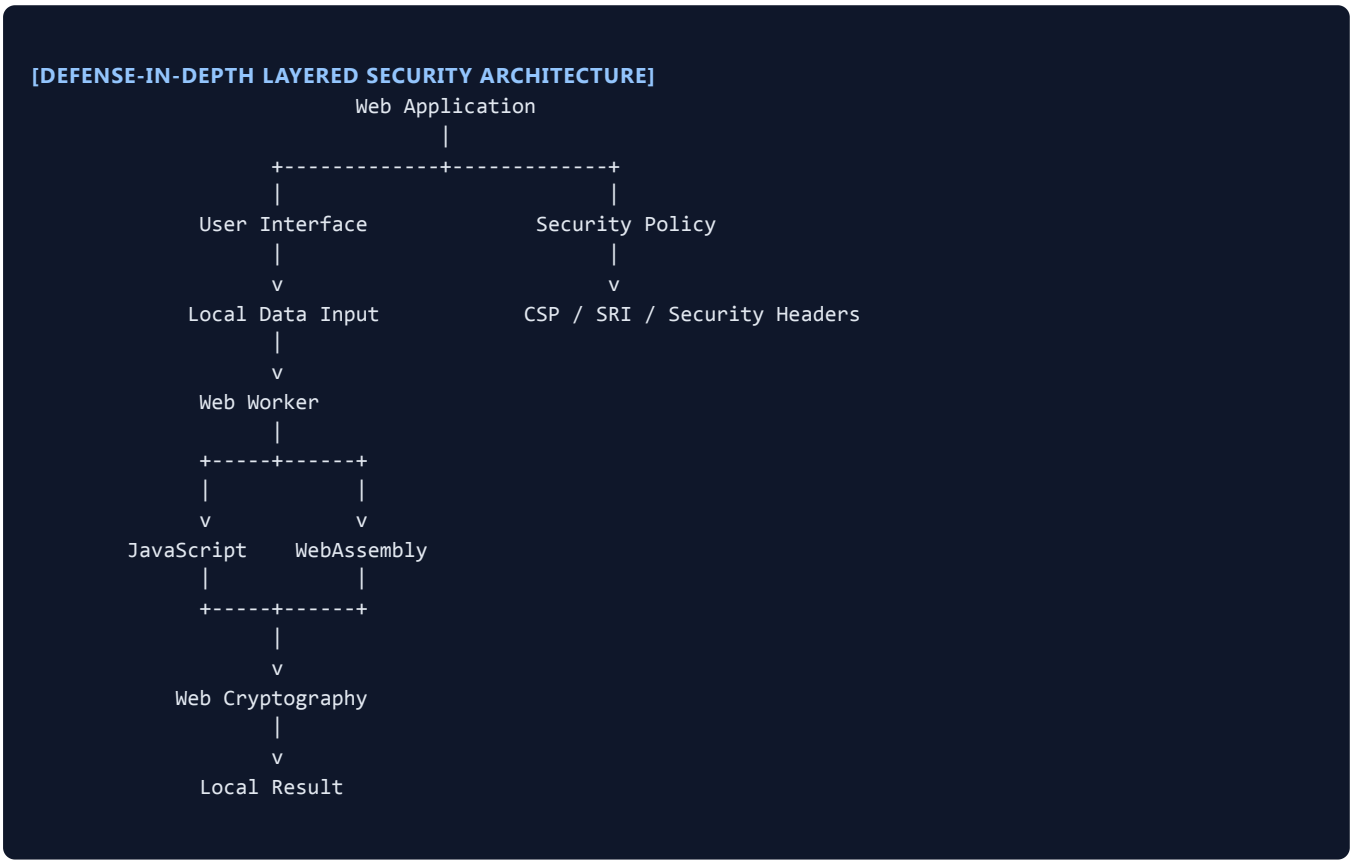
A structured approach to privacy analysis models the complete lifecycle of user information across discrete processing stages:

- **Stage 1 — Acquisition:** The user selects a local file through file picker or drag-and-drop.
- **Stage 2 — Representation:** The browser instantiates in-memory representation (ArrayBuffer, Blob).
- **Stage 3 — Processing:** The application performs computation entirely within browser memory.
- **Stage 4 — Transformation:** The engine produces a transformed output binary stream.
- **Stage 5 — Export:** The user downloads the final output via an ephemeral Blob URL.
- **Stage 6 — Disposal:** Temporary references are revoked and buffer memory is reclaimed.



12. Security Architecture

A privacy-oriented browser application must combine multiple interlocking security layers rather than relying on local execution alone:



This architecture follows a rigorous defense-in-depth philosophy. No individual technology should be treated as the sole security control.

13. The Role of Content Security Policy

Content Security Policy (CSP) can reduce the browser application's exposure to certain classes of script-injection attacks. A restrictive policy controls which scripts may execute, where scripts can be loaded from, whether objects can be embedded, which network destinations may be contacted, and other browser behaviors.

OWASP describes CSP as a defense-in-depth mechanism particularly relevant to XSS and client-side attacks [5]. For a privacy-oriented application, a restrictive network policy (e.g. `connect-src 'none'` or strictly whitelisted origin) also helps make unintended data transmission readily detectable during automated security audits. However, CSP must be regarded as one layer of a larger security architecture rather than a substitute for secure coding practices.

14. Dependency Management

A particularly important issue in client-side privacy applications is the software supply chain. When an application uses a third-party JavaScript library, that library executes within the user's browser environment and may potentially interact with application data according to the privileges available to it.

A privacy-focused architecture should therefore implement:

- Strict minimization of external third-party dependencies;
- Rigorous version pinning and lockfile auditing;
- Subresource Integrity (SRI) hashes for all static assets;
- Elimination of third-party tracking, profiling, and ad scripts;
- Regular vulnerability scanning of package manifests; and
- Maintenance of a formal Software Bill of Materials (SBOM).

The goal is to maintain a minimal, auditable Trusted Computing Base (TCB).

15. Privacy Does Not Mean Complete Security

It is essential to distinguish between discrete security properties and avoid unwarranted assumptions:

Local processing can effectively mitigate:

- Unnecessary file uploads over the public network;
- Server-side copies and unauthorized persistence;
- Interception of sensitive payload transit;
- Centralized database breaches; and
- Third-party cloud infrastructure access.

Local processing does NOT automatically mitigate:

- Hostile or rogue browser extensions installed by the user;
- Endpoint malware, keyloggers, or operating system compromise;
- Malicious dependencies injected into the application bundle;
- DOM-based Cross-Site Scripting (XSS);
- Zero-day browser sandbox vulnerabilities; or
- Screen capture, shoulder surfing, or unauthorized physical device access.

> **“The file never leaves the browser” must only be asserted when technically verified through continuous network instrumentation.**

16. Verification of the No-Upload Property

A valuable engineering practice is independent verification of data-boundary integrity. Developers and auditors can inspect network activity using browser developer tools, automated test harnesses, and packet inspection proxies.

Evaluation should audit HTTP/HTTPS requests, request payloads, POST/PUT bodies, WebSocket framing, fetch and XHR invocations, third-party requests, and background telemetry. A file-processing application should be tested using deliberately

identifiable test markers:

```
LOCAL_TEST_IDENTIFIER_2026_001
```

Network logs are analyzed to prove that the test marker never surfaces in outbound request headers, query strings, or request bodies.

17. Experimental Evaluation Framework

A rigorous evaluation framework measures performance and privacy metrics under controlled, reproducible conditions across discrete test matrices:

Independent Variables: File size, MIME type, processing algorithm, browser engine (Chromium, Gecko, WebKit), hardware class, thread allocation, and WebAssembly acceleration.

Dependent Variables: Execution latency, peak memory allocation, CPU utilization, UI frame rate (jank), outbound network bytes (which must evaluate to zero), and completion reliability.

Experimental topologies compare:

- **Configuration A:** Main Execution Thread + Pure JavaScript
- **Configuration B:** Main Thread + Web Worker + Pure JavaScript
- **Configuration C:** Main Thread + Web Worker + WebAssembly SIMD Acceleration

18. Performance Evaluation

Performance in browser-native systems must be evaluated across multiple dimensions:

Evaluation Dimension	Primary Research Question	Target Benchmark Threshold
Processing Latency	How long does local computation take relative to server-side round-trips?	≤ 1.5× Native Desktop Speed
Peak Memory Footprint	How much volatile RAM is required during peak transformation?	≤ 2.5× Input File Size
Main Thread Responsiveness	Does background execution maintain 60 FPS user interface rendering?	0 Frame Drops / Zero Jank
Outbound Network Traffic	Does processing transmit any portion of file data to external hosts?	0 Bytes Transmitted
Mobile Thermal & Battery Load	How resource-efficient is computation on constrained mobile processors?	Minimal Energy Consumption
Job Completion Reliability	Does the pipeline successfully complete without browser tab OOM crashes?	> 99.9% Execution Success

19. Privacy–Performance Trade-Off

Local processing shifts the physical location of computational work. Instead of transmitting files across the network to server clusters, computation is absorbed by the user's local processor. While this maximizes privacy and eliminates cloud transmission latency, it requires sufficient client hardware resources.

On high-performance desktop machines, client-side execution is often instantaneous. On low-power mobile devices, however, processing large multimedia files can create battery draw, memory pressure, or execution throttling. Architectural design must therefore balance computational demand against client device capabilities.

20. Offline and Online Modes

An additional advantage of client-side execution is native support for offline processing. When web application assets (HTML, scripts, WASM binaries) are cached via Service Workers, operations can proceed without an active network connection.

However, offline capability and client-side processing remain distinct architectural concepts. An application may operate locally while still requiring network access for authentication or telemetry. The system architecture must explicitly specify and document all network communications.

21. Applications in Information Security

Client-side processing provides unique advantages across core security workflows:

- **File Integrity Verification:** Independent client-side cryptographic hashing prior to external transmission.
- **Secure Document Transformation:** PDF redaction, conversion, and metadata scrubbing without exposing raw content to remote hosting providers.
- **Local Security Analysis:** In-browser static inspection of document structures, embedded scripts, and anomalous metadata.
- **Privacy-Preserving Data Preparation:** Preprocessing and anonymizing sensitive datasets locally before selective sharing.
- **Client-Side Cryptographic Sanitation:** Irreversible sanitization of binary artifacts in compliance with data sovereignty regulations.

22. Limitations

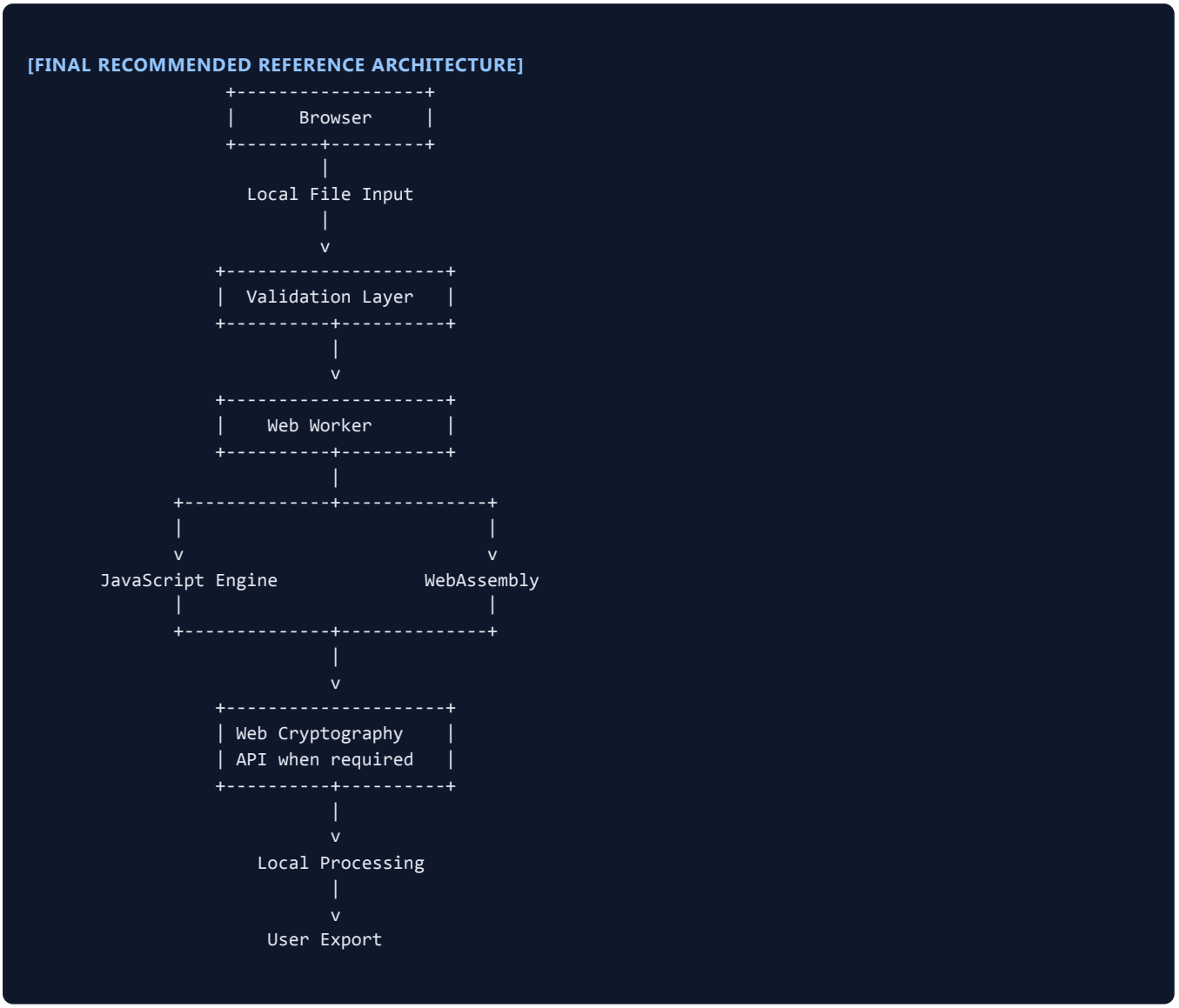
The proposed architecture exhibits known constraints. First, massive workloads requiring distributed compute clusters or extensive machine-learning parameter spaces remain better suited to server infrastructure. Second, browser sandbox memory ceilings restrict single-job file sizes. Third, application security remains fundamentally dependent on the integrity of the delivering web server and CDN. Fourth, client-side execution cannot protect against local endpoint malware. Fifth, client storage must be strictly managed to prevent data residue. Finally, cryptographic primitives must be implemented with rigorous algorithm parameter fidelity [1, 8].

23. Discussion

The transition from cloud-centric processing toward client-side computation represents an important architectural paradigm shift. Rather than defaulting to remote servers, modern software engineering can prioritize data sovereignty: *Does this information actually need to leave the client device?* When local processing is feasible, unnecessary data movement can be eliminated at the architectural level, providing robust privacy guarantees that are inherent by design rather than retrofitted through administrative policies.

24. Recommended Architecture

A reference implementation of a privacy-oriented browser platform combines local input acquisition, a strict validation boundary, background Web Workers, WebAssembly computation modules, and Web Cryptography interfaces, ensuring that the application server remains completely detached from sensitive user data:



25. Future Research

Promising research directions include: (1) Automated formal verification tools that statically prove zero-egress data boundaries in web apps; (2) Standardized browser-native privacy indicators signaling when a page processes data entirely locally; (3) Memory safety isolation mechanisms for WebAssembly modules handling classified documents; (4) Energy-optimized compilation profiles for mobile client execution; (5) Cryptographic multi-party hybrid computation models; and (6) Privacy-preserving client-side machine learning inference.

26. Conclusion

This paper examined a browser-based architecture for privacy-preserving information processing in which sensitive data remains strictly confined to the user's device during computational operations. Modern web technologies—including Web

Workers, WebAssembly, and the Web Cryptography API—provide a mature, performant foundation for client-side document and data manipulation [1, 2, 3].

The principal architectural benefit is rigorous data minimization: by executing transformations locally, the application avoids transmitting sensitive files across third-party networks and remote servers. While client-side processing is not a panacea for all endpoint security challenges, it establishes a foundational design pattern for privacy-first web systems: *transmit only what is strictly necessary, and process everything else locally.*

References

[1] World Wide Web Consortium (W3C). *Web Cryptography API / Web Cryptography Level 2*. W3C Web Cryptography Working Group, W3C Recommendation. Available: https://www.w3.org/TR/WebCryptoAPI/

[2] World Wide Web Consortium (W3C). *Web Workers*. W3C / WHATWG Web Platform Specifications. Available: https://html.spec.whatwg.org/multipage/workers.html

[3] WebAssembly Community Group. *WebAssembly Core Specification — Version 2.0*. WebAssembly.org, W3C Recommendation. Available: https://webassembly.github.io/spec/core/

[4] OWASP Foundation. *OWASP Top 10 Client-Side Security Risks*. Open Web Application Security Project. Available: https://owasp.org/

[5] OWASP Foundation. *Content Security Policy Cheat Sheet*. OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/

[6] OWASP Foundation. *HTML5 Security Cheat Sheet*. OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/

[7] OWASP Foundation. *Web Security Testing Guide — Testing Browser Storage*. OWASP WSTG-CLNT-02. Available: https://owasp.org/

[8] W3C Security Interest Group. *Cryptography Usage in Web Standards*. W3C Group Draft Note. Available: https://www.w3.org/

[9] Utilvo. (2026). *Utilvo: Web application for client-side document processing and file conversion*. Available: https://utilvo.com/

islam ayoub

Appendix A: Client-Side PDF Processing Reference Engines

All reference tools below execute 100% locally in browser memory via WebAssembly & HTML5 Canvas:

Utilvo Main Platform:	https://utilvo.com/
Utilvo PDF Suite Hub:	https://utilvo.com/pdf-tools
Advanced PDF Editor Studio:	https://utilvo.com/pdf-tools/advanced-editor
Find & Replace PDF Text:	https://utilvo.com/pdf-tools/find-and-replace-pdf-text
Whiteout & Redact PDF:	https://utilvo.com/pdf-tools/whiteout-pdf
Draw & Highlight on PDF:	https://utilvo.com/pdf-tools/draw-on-pdf
Add Image & Stamp to PDF:	https://utilvo.com/pdf-tools/add-image-to-pdf
Insert Blank Page in PDF:	https://utilvo.com/pdf-tools/insert-blank-page-pdf
Change PDF Background Color:	https://utilvo.com/pdf-tools/change-pdf-background-color
Merge PDF Documents:	https://utilvo.com/pdf-tools/merge-pdf
Split & Extract PDF Pages:	https://utilvo.com/pdf-tools/split-pdf
Compress PDF Size Locally:	https://utilvo.com/pdf-tools/compress-pdf